

Lo primero que deberemos hacer es tener un editor de código e instalar python, en mi caso voy a usar el visual studio code e instalar python desde dentro del mismo. Con esto empezamos con el primer script (habrá varios antes de llegar al definitivo, <https://github.com/blxxdyclxck>):

```
1  import subprocess
2
3  interface = input("interface > ")
4  newMac = input("mac > ")
5
6  print("[+] Cambiando direccion MAC para " + interface + " a " + newMac)
7
8  subprocess.call("ifconfig " + interface + " down", shell=True)
9  subprocess.call("ifconfig " + interface + " hw ether " + newMac, shell=True)
10 subprocess.call("ifconfig " + interface + " up", shell=True)
11
12
```

Este es un script cuyo objetivo es cambiar la MAC de una interfaz de red, solicitando al usuario tanto la interfaz de la MAC que queremos cambiar como la MAC nueva. Vamos a ir explicando por partes:

1. import subprocess

Con import lo que hacemos es importar módulos para trabajar luego en el propio script, en este caso el módulo subprocess permite la ejecución de comandos del sistema desde python, aquí lo usaremos para ejecutar comandos de red

2. interface = input("interface > ")

Aquí lo que hacemos es declarar la variable interface, la cual contiene una función input(esta se usa para recibir la entrada de un usuario por terminal) en este caso el mensaje de este input es interface > es decir se le pide al usuario que ingrese la interfaz que quiera cambiar y esta se almacenará en la variable interface.

3. newMac = input("mac > ")

Igual que en la línea anterior en esta declaramos una variable que usa la función input pero en este caso lo que almacenamos no es la interfaz que el usuario quiere modificar si no la nueva MAC que se quiere usar.

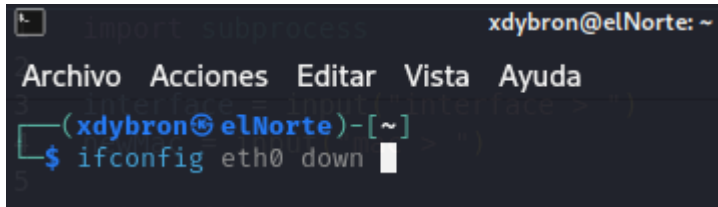
4. print("[+] Cambiando direccion MAC para " + interface + " a " + newMac)

Con la función print lo que estamos haciendo es mostrar por pantalla al usuario un mensaje, el cual seria: [+] Cambiando direccion MAC para "interface" a "newMac" tanto interface como newMac varían segun lo que el usuario escriba por la interfaz ej: [+] Cambiando direccion MAC para eth0 a 00:11:22:33:44:55

5. subprocess.call("ifconfig " + interface + " down", shell=True)

Como dijimos antes subprocess es un módulo de python que nos permite ejecutar comandos del sistema, con el .call lo que se consigue es que se espere a que termine este comando para continuar. la parte de "ifconfig " + interface + " down" es el comando que

ejecuta el subprocess, ifconfig sirve para gestión de interfaces de red, interface es la variable donde se nos guarda la interfaz a modificar y down es la orden que damos para desactivar esta interfaz y así poder modificarla. Esto lo hacemos así ya que la mayoría de las interfaces no nos permiten cambiar la MAC si están activas, este conjunto sería lo mismo que en terminal poner por ejemplo:

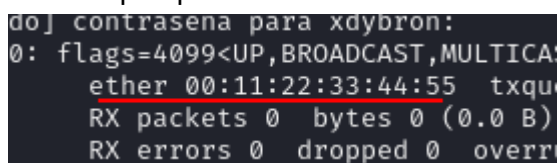


```
xddybron@elNorte: ~  
Archivo Acciones Editar Vista Ayuda  
(xddybron@elNorte)-[~]  
$ ifconfig eth0 down
```

La parte de **shell=True** lo que hace este argumento es indicar que se ejecutará en la terminal o shell del sistema, python ejecuta el comando como si lo escribieras directamente en terminal, esto es muy peligroso pues como explicaré más abajo esto abre un fallo de seguridad ya que nos permite la ejecución de código y al no estar protegido el código podemos inyectar otros comandos fácilmente.

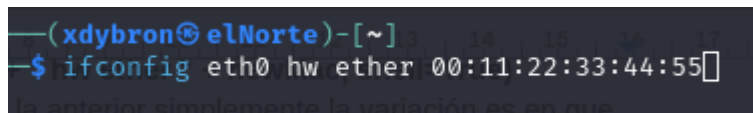
6. subprocess.call("ifconfig " + interface + " hw ether " + newMac, shell=True)

En esta línea hacemos casi lo mismo que en la anterior simplemente la variación es en que en vez de usar el argumento down usamos hw ether, hw(hardware) nos indica que queremos modificar un atributo de hardware de la interfaz de red, y ether nos indica que ese atributo que queremos cambiar es la dirección MAC.



```
do] contraseña para xddybron:  
0: flags=4099<UP,BROADCAST,MULTICAST>  
    ether 00:11:22:33:44:55 txque  
    RX packets 0 bytes 0 (0.0 B)  
    RX errors 0 dropped 0 overr
```

La parte de newMac es la variable donde está almacenada la nueva dirección MAC que el usuario a introducido por ejemplo: 00:11:22:33:44:55; al final esta línea sería como poner en terminal:



```
(xddybron@elNorte)-[~]  
$ ifconfig eth0 hw ether 00:11:22:33:44:55
```

7. subprocess.call("ifconfig " + interface + " up", shell=True)

En esta línea nuevamente hacemos casi lo mismo que en las dos anteriores, en esta la única variación es que en vez de hacer down para desactivar la interfaz como en la primera línea con subprocess, en esta haremos up para volver a activar la interfaz ya que esta ya ha sido modificada.

IMPORTANTE:

Este script no tiene en cuenta los privilegios del usuario que lo este lanzando es decir que si no tenemos privilegios de root no cambiara la MAC ya que el propio ifconfig no funciona sin privilegios:

```
(xdybron@elNorte)-[~/Escritorio/pythonMalware]
$ sudo ifconfig eth0
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 00:11:22:33:44:55 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(xdybron@elNorte)-[~/Escritorio/pythonMalware]
$ python macChanger3.py
interface > eth0
mac > 00:11:22:44:55:66
[+] Cambiando direccion MAC para eth0 a 00:11:22:44:55:66
SIOCSIFFLAGS: Operación no permitida
SIOCSIFHWADDR: Operación no permitida
SIOCSIFFLAGS: Operación no permitida

(xdybron@elNorte)-[~/Escritorio/pythonMalware]
$ sudo ifconfig eth0
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 00:11:22:33:44:55 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(xdybron@elNorte)-[~/Escritorio/pythonMalware]
$
```

```
$ sudo ifconfig eth0
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 00:11:22:33:44:55 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(xdybron@elNorte)-[~/Escritorio/pythonMalware]
$ sudo python macChanger3.py
interface > eth0
mac > 00:11:33:44:55:66
[+] Cambiando direccion MAC para eth0 a 00:11:33:44:55:66

(xdybron@elNorte)-[~/Escritorio/pythonMalware]
$ sudo ifconfig eth0
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 00:11:33:44:55:66 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Como mencione antes un problema de este script es que si nos fijamos en el shell=True nos daremos cuenta de que podemos ejecutar comandos para los que no esta pensado este script veamos un ejemplo:

```
(xdybron@elNorte)-[~/Escritorio/pythonMalware]
$ sudo python macChanger3.py
interface > eth0;ls -las;pwd
```

Con el ; lo que indicamos es que termina el comando y empieza otro

Vamos a lanzar otros dos comandos para ver que nos responde

```
$ sudo python macChanger3.py
interface > eth0;ls -las;pwd
mac > 00:22:11:44:33:77
[+] Cambiando direccion MAC para eth0;ls -las;pwd a 00:22:11:44:33:77
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 00:11:33:44:55:66 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

total 16
4 drwxrwxr-x 3 xdybron xdybron 4096 mar 13 17:41 .script veamos un ejemplo:
4 drwxr-xr-x 3 xdybron xdybron 4096 mar 13 16:40 ..
4 drwxrwxr-x 2 xdybron xdybron 4096 mar 13 17:41 img
4 -rwxrwxrwx 1 xdybron xdybron 355 mar 13 17:38 macChanger3.py
/home/xdybron/Escritorio/pythonMalware
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 00:11:33:44:55:66 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

total 16
4 drwxrwxr-x 3 xdybron xdybron 4096 mar 13 17:41 .
4 drwxr-xr-x 3 xdybron xdybron 4096 mar 13 16:40 ..
4 drwxrwxr-x 2 xdybron xdybron 4096 mar 13 17:41 img
4 -rwxrwxrwx 1 xdybron xdybron 355 mar 13 17:38 macChanger3.py
/home/xdybron/Escritorio/pythonMalware
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 00:11:33:44:55:66 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

total 16
4 drwxrwxr-x 3 xdybron xdybron 4096 mar 13 17:41 .
4 drwxr-xr-x 3 xdybron xdybron 4096 mar 13 16:40 ..
4 drwxrwxr-x 2 xdybron xdybron 4096 mar 13 17:41 img
```

Como podemos ver el programa se lanza sin problema ninguno.

Aquí ya podemos ver como nos sale el resultado del segundo comando que le he pedido es decir del ls -las

```
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

total 16
4 drwxrwxr-x 3 xdybron xdybron 4096 mar 13 17:41 .
4 drwxr-xr-x 3 xdybron xdybron 4096 mar 13 16:40 ..
4 drwxrwxr-x 2 xdybron xdybron 4096 mar 13 17:41 img
4 -rwxrwxrwx 1 xdybron xdybron 355 mar 13 17:38 macChanger3.py
/home/xdybron/Escritorio/pythonMalware

(xdybron@elNorte)-[~/Escritorio/pythonMalware]
```

Como vemos se lleva a cabo sin problema, esto es malo ya que lo que hacemos es crear una vulnerabilidad y dar la posibilidad de hacer una ejecución de comandos no deseada.

```
8
9 subprocess.call(["ifconfig", interface, "down"])
10 subprocess.call(["ifconfig", interface, "hw", "ether", newMac])
11 subprocess.call(["ifconfig", interface, "up"])
```

Aquí lo que hacemos es en vez de usar una cadena de texto lo que estamos haciendo es pasar cada argumento por separado como elemento de una lista de esta manera python ejecuta sin pasar por una shell ya que al usar lista en vez de cadena de texto no es necesario procesar el texto en una shell, y además ejecuta directamente el binario del comando /sbin/ifconfig.

Prueba:

```
(xdybron@elNorte)-[~/Escritorio/pythonMalware]
$ sudo python macChanger4.py
[sudo] contraseña para xdybron:
interface > eth0;ls;pwd
mac > 00:11:22:33:44:44
[+] Cambiando direccion MAC para eth0;ls;pwd a 00:11:22:33:44:44
eth0;ls;pwd: ERROR while getting interface flags: No existe el dispositivo
SIOCSIFHWADDR: No existe el dispositivo
eth0;ls;pwd: ERROR while getting interface flags: No existe el dispositivo

(xdybron@elNorte)-[~/Escritorio/pythonMalware]
$ sudo ifconfig eth0
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
ether 00:11:33:44:55:66 <que elen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(xdybron@elNorte)-[~/Escritorio/pythonMalware]
$ sudo python macChanger4.py
interface > eth0
mac > 00:11:22:33:44:44
[+] Cambiando direccion MAC para eth0 a 00:11:22:33:44:44
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
ether 00:11:22:33:44:44 <que elen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

(xdybron@elNorte)-[~/Escritorio/pythonMalware]
```

De esta manera obtenemos el mismo resultado evitando la inyección de comandos